

RTX 5090 FP8/FP6/FP4 Reduction Tree Width Report

Direct PTX tensor-core boundary measurements

Date: 2026-05-09

Headline finding. All tested FP8, FP6, and FP4 direct-PTX QMMA variants expose a **21-bit effective addition/reduction boundary** for the m16n8k32 probe. The binary32 reference for the same probe is **24 bits**, so the measured low-precision tensor-core path is **3 effective bits narrower** in this test.

GPU	NVIDIA GeForce RTX 5090, compute capability 12.0
CUDA	Toolkit 13.2, nvcc V13.2.78
Build target	compute_120a / sm_120a
Source	src/tc_test_numerics-5090-lowp-reduction-width.cu

1 Scope

The tests execute inline PTX `mma.sync.aligned` instructions directly from a single warp. They do not call cuBLASLt or any GEMM library. The reported quantity is an externally observable effective addition/reduction width for one `m16n8k32` instruction. It should not be read as a complete physical map of the internal reduction tree.

FP8 uses regular unscaled FP8 MMA instructions. FP6 and FP4 use the PTX `kind::f8f6f4` form. Compact block-scaled MXFP4/MXFP6 instructions are outside this report.

2 Methodology

Each measurement runs one direct tensor-core MMA:

$$D = C + \sum_{i=0}^{31} A_i B_i$$

All A and B elements are encoded as exactly 1.0. The exact dot-product contribution is therefore $K = 32$. The C accumulator is a binary32 value set to 2^{shift} . The probe scans shifts from 16 to 40 and records the largest shift where the output is still greater than the base accumulator.

$$p = max_changed_shift - \log_2(K) + 1$$
$$K = 32, \quad \log_2(K) = 5, \quad max_changed_shift = 25, \quad p = 21$$

A full binary32 significand has 24 bits. For the same $K = 32$ boundary probe, the binary32 reference max shift is:

$$\log_2(32) + 24 - 1 = 28$$

3 Key Code Snippets

The constants define the K dimension, scan interval, and binary32 reference.

```
static const int kInstructionK = 32;
static const int kLog2InstructionK = 5;
static const int kMinShift = 16;
static const int kMaxShift = 40;
static const int kExpectedBinary32Bits = std::numeric_limits<float>::digits;
static const int kExpectedBinary32MaxShift =
    kLog2InstructionK + kExpectedBinary32Bits - 1;
```

FP4 E2M1 values are shifted into the central four bits required by the `kind::f8f6f4` PTX operand format.

```
static std::uint8_t encode_fp4_e2m1_padded(float value) {
    return static_cast<std::uint8_t>((__nv_fp4_e2m1(value).__x << 2));
}
```

The MMA path is direct inline PTX. The output operands are binary32 registers.

```
#define TCNB_MMA_ASM(instruction) \
asm volatile(instruction " {%0, %1, %2, %3}, " \
```

```

        "{%4, %5, %6, %7}, {%8, %9}, "
        "{%10, %11, %12, %13};\n"
: "=f"(d0), "=f"(d1), "=f"(d2), "=f"(d3)
: "r"(a_pack), "r"(a_pack), "r"(a_pack), "r"(a_pack),
  "r"(b_pack), "r"(b_pack), "f"(base), "f"(base),
  "f"(base), "f"(base)
\
\
\
\
\

```

The scan records the last visible contribution and converts it to bits.

```

for (int shift = kMinShift; shift <= kMaxShift; ++shift) {
    float base = std::ldexp(1.0f, shift);
    float sample = run_once<variant>(a_pack, b_pack, shift, device_out);

    if (sample > base) {
        result.max_changed_shift = shift;
        result.sample_at_max = sample;
    } else if (result.max_changed_shift >= 0 &&
               result.sample_after_max == 0.0f) {
        result.sample_after_max = sample;
    }
}

result.observed_bits =
    result.max_changed_shift - kLog2InstructionK + 1;

```

The build targets the accelerated SM120a feature set explicitly.

```

test-5090-fp8-reduction-width: $(SRC_5090_LOWP_REDUCTION_WIDTH)
$(NVCC) $(NVCC_INCLUDES) -DTCNB_REDUCTION_FP8 -o $@ \
    -arch=$(GPU_COMPUTE_5090_ACCEL) -code=$(GPU_SM_5090_ACCEL) \
    $(NVCC_STD) $<

```

4 Instructions Tested

Family	Variant	PTX instruction
FP8	E4M3 x E4M3	mma.sync.aligned.m16n8k32.row.col.f32.e4m3.e4m3.f32
FP8	E5M2 x E5M2	mma.sync.aligned.m16n8k32.row.col.f32.e5m2.e5m2.f32
FP8	E4M3 x E5M2	mma.sync.aligned.m16n8k32.row.col.f32.e4m3.e5m2.f32
FP8	E5M2 x E4M3	mma.sync.aligned.m16n8k32.row.col.f32.e5m2.e4m3.f32
FP6	E2M3 x E2M3	mma.sync.aligned.m16n8k32.row.col.kind::f8f6f4.f32.e2m3.e2m3.f32
FP6	E3M2 x E3M2	mma.sync.aligned.m16n8k32.row.col.kind::f8f6f4.f32.e3m2.e3m2.f32
FP4	E2M1 x E2M1	mma.sync.aligned.m16n8k32.row.col.kind::f8f6f4.f32.e2m1.e2m1.f32

5 Results

Core numbers

Largest shift where $D > C$	25
Observed effective width	21 bits
Binary32 reference max shift	28
Binary32 reference width	24 bits
Gap from binary32 reference	-3 bits

Family	Variant	Max shift	Sample at max	First unchanged	Bits
FP8	E4M3 x E4M3	25	33,554,464	67,108,864	21
FP8	E5M2 x E5M2	25	33,554,464	67,108,864	21
FP8	E4M3 x E5M2	25	33,554,464	67,108,864	21
FP8	E5M2 x E4M3	25	33,554,464	67,108,864	21
FP6	E2M3 x E2M3	25	33,554,464	67,108,864	21
FP6	E3M2 x E3M2	25	33,554,464	67,108,864	21
FP4	E2M1 x E2M1	25	33,554,464	67,108,864	21

At shift 25, the base is $2^{25} = 33,554,432$, and the observed sample is 33,554,464. The +32 contribution is still visible. At shift 26, the first unchanged sample is 67,108,864, equal to 2^{26} . The contribution is no longer visible.

6 Facts and Evidence

Fact	Evidence
Direct PTX path	The source emits inline <code>mma.sync.aligned</code> ; no cuBLASLt dependency is present.
Single-instruction boundary	Each sample launches one warp and one <code>m16n8k32</code> MMA for the selected variant.
Exact input contribution	All A and B operands encode 1.0; the K dimension contributes exactly 32.
Uniform observed width	All FP8, FP6, and FP4 result files report max shift 25 and width 21.
Binary32 comparison	Binary32 would produce max shift 28 for the same K and boundary formula.

6.1 SASS Confirmation

Target	Confirmed SASS mnemonics
test-5090-fp8-reduction-width	QMMMA.16832.F32.E4M3.E4M3, QMMMA.16832.F32.E5M2.E5M2, QMMMA.16832.F32.E4M3.E5M2, QMMMA.16832.F32.E5M2.E4M3
test-5090-fp6-reduction-width	QMMMA.16832.F32.E2M3.E2M3, QMMMA.16832.F32.E3M2.E3M2
test-5090-fp4-reduction-width	QMMMA.16832.F32.E2M1.E2M1

7 Reproduction

```
make -B test-5090-fp8-reduction-width \
      test-5090-fp6-reduction-width \
      test-5090-fp4-reduction-width

python3 run_tests.py -o 5090 \
      5090-fp8-reduction-width \
      5090-fp6-reduction-width \
      5090-fp4-reduction-width

cuobjdump --dump-sass test-5090-fp8-reduction-width
cuobjdump --dump-sass test-5090-fp6-reduction-width
cuobjdump --dump-sass test-5090-fp4-reduction-width
```

8 Interpretation

The root cause of the 24-bit binary32 expectation failure is the low-precision QMMA accumulation/reduction path itself. The inputs are exact, the dot-product contribution is exact, and SASS confirms direct tensor-core QMMA instructions. The measured path simply stops preserving the +32 contribution three shifts earlier than a full binary32 significand path would.

The result is consistent across all tested FP8, FP6, and FP4 input type combinations: the observable effective addition/reduction boundary is 21 bits.

9 Limitations

- This is a single-instruction m16n8k32 boundary probe, not a full GEMM accuracy characterization.
- The measurement reports observable effective width, not the internal physical topology of the hardware reduction tree.
- FP6 and FP4 are tested through the PTX kind::f8f6f4 form. Compact block-scaled MXFP4/MXFP6 instructions are not covered.
- The input pattern is all ones. Other value distributions can exercise different rounding and normalization behavior.

10 Summary

RTX 5090 direct-PTX FP8, FP6, and FP4 tensor-core MMA instructions tested here all measure at 21 effective reduction/addition bits for the `m16n8k32` boundary probe. The `binary32` reference is 24 bits. The observed low-precision QMMA path is therefore consistently 3 effective bits narrower than `binary32` for this measurement.